# SE3082

## AWS EC2 Lab - Screenshots and Calculations

*Lab 08*

*Exercise 3: Parallel Monte Carlo Pi using MPI*

***IT24610328***

## On instance with 2 vCPUs (t3.micro)

### Nano - creating the Monte Carlo Pi code

```
  GNU nano 6.2                                                                pi_mpi.cc
#include <cstdio>
#include <cstdlib>
#include <mpi.h>
int main(void)
{
    int rank, size;
    MPI_Status status;
    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // IT24610328 - Ex3 - Added total points and per process share
    const long long N = 10000000;
    long long mine = N / size;
    if (rank == size - 1) mine += N % size;

    // IT24610328 - Ex3 - Added barrier and timer
    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();

    // IT24610328 - Ex3 - Added Monte Carlo loop with a different seed per rank
    unsigned int seed = 12345 + rank;
    long long hits = 0;
    for (long long i = 0; i < mine; i++) {
        double x = (double)rand_r(&seed) / RAND_MAX;
        double y = (double)rand_r(&seed) / RAND_MAX;
        if (x * x + y * y <= 1.0) hits++;
    }

    // IT24610328 - Ex3 - Added send of hit counts to rank 0 and receive from each rank in order
    if (rank != 0) {
        MPI_Send(&hits, 1, MPI_LONG_LONG, 0, 0, MPI_COMM_WORLD);
    } else {
        long long total = hits, other;
        for (int src = 1; src < size; src++) {
            MPI_Recv(&other, 1, MPI_LONG_LONG, src, 0, MPI_COMM_WORLD, &status);
            total += other;
        }
        double t1 = MPI_Wtime();
        printf("Processes = %d\n", size);
        printf("Pi = %.6f\n", 4.0 * total / N);
        printf("Time = %f seconds\n", t1 - t0);
    }
    MPI_Finalize();
}
```

Output from running the Monte Carlo Pi code, 10,000,000 points (4 processes)

```
[ec2-user@ip-172-31-17-201 ~]$ mpic++ pi_mpi.cc -o pi_mpi
[ec2-user@ip-172-31-17-201 ~]$ mpirun --oversubscribe -np 4 ./pi_mpi
Processes = 4
Pi = 3.140876
Time = 0.170875 seconds
```